COMP2511

Tutorial 7

Last Tutorial

- Streams
- Strategy Pattern
- Observer Pattern

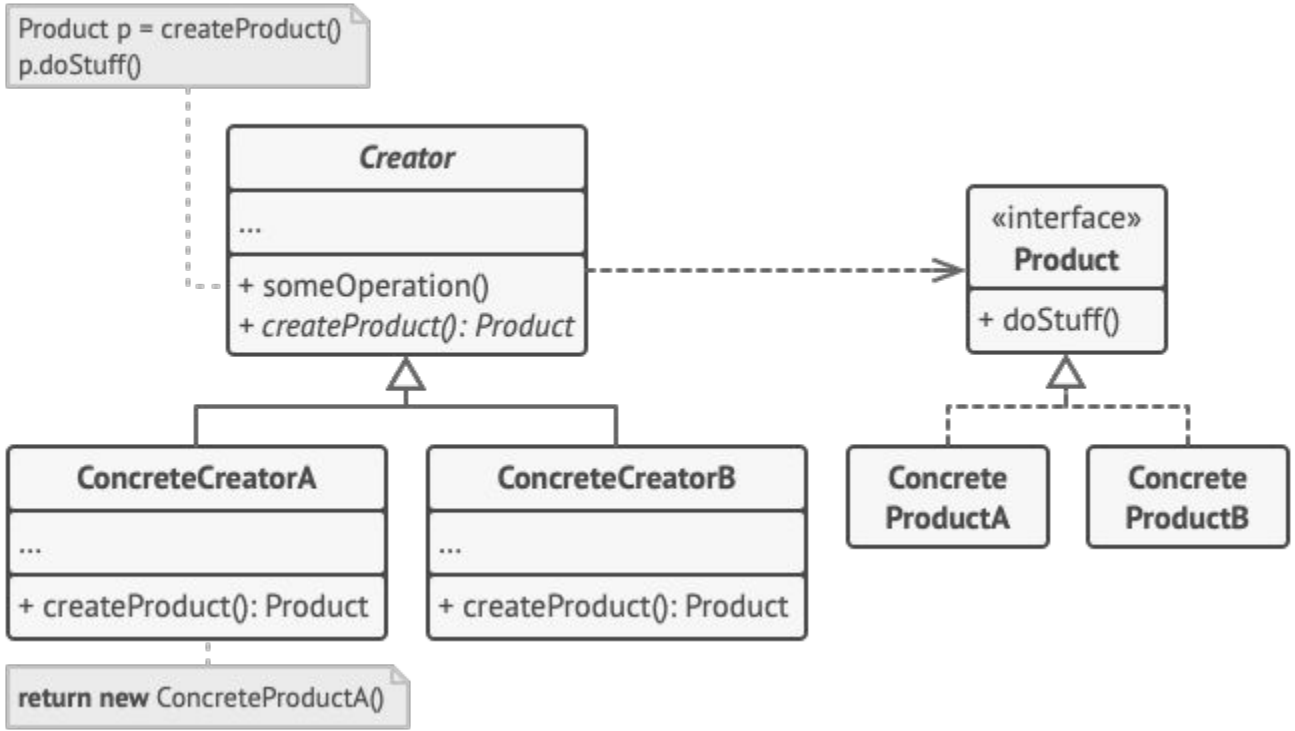
This Tutorial

- Abstract Factory Pattern
- Decorator Pattern
- Singleton Pattern

Factory Pattern (again)

- **For the purposes of this course, think of the Factory Pattern as simply moving construction logic to a separate "factory" class.**
- This is different to how the Factory Pattern is presented elsewhere.
- In the more typical definition of the Factory Pattern, factories are used to create objects *without specifying their concrete types*.

Factory Pattern: Implementation



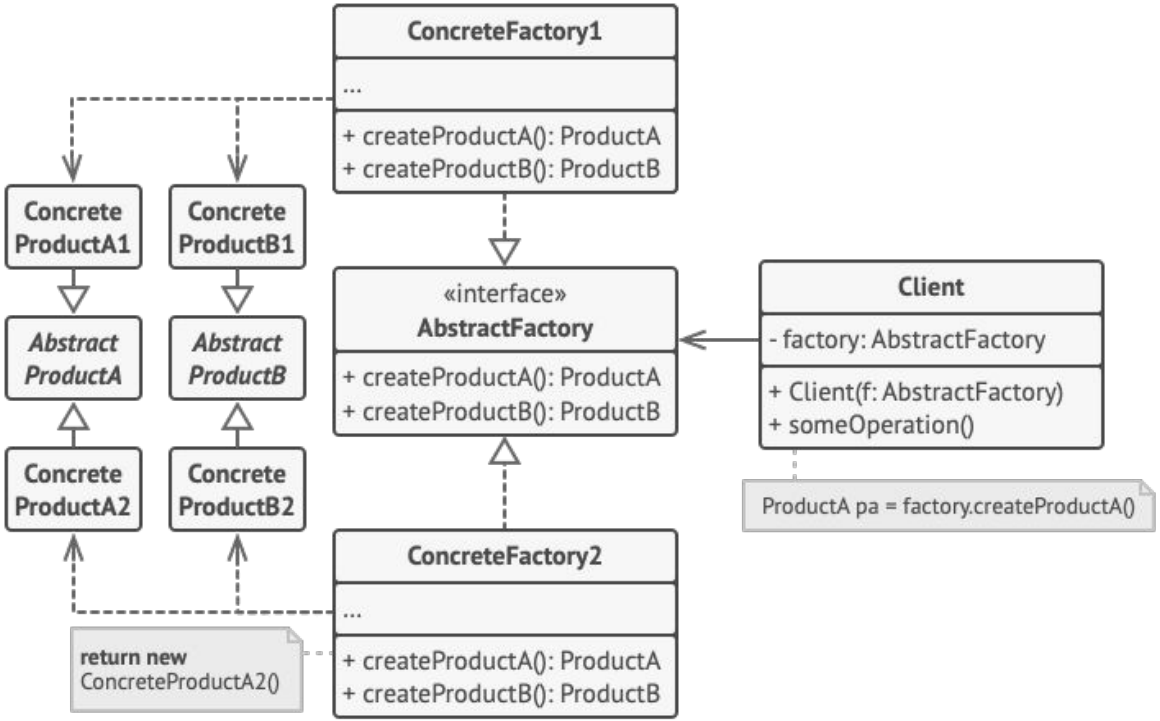
Abstract Factory Pattern

- The Abstract Factory Pattern is a (creational) design pattern that provides interfaces to create **families of related objects** without specifying their concrete classes.
  - This is very similar to the Factory Pattern. The difference is that the Abstract Factory pattern comes into play when the objects you are creating have different *categories*.
- The Abstract Factory Pattern suggests that you define a distinct factory for each category, all implementing a common interface containing the different objects that each factory should be able to produce.
  - This ensures that related objects are kept together and makes changing between different categories very easy (just change the factory being used).

Abstract Factory Pattern: Example

- Consider shirts, pants and jackets. These are related objects (clothing) which can be further classified into different categories like colour, size, brand etc.
- If you were categorising clothing by colour, you could have a `ClothingFactory` interface with `createShirt()`, `createPants()`, `createJacket()` methods, and concrete `RedClothingFactory`, `GreenClothingFactory` etc. classes which each provide their own implementation of those methods.

Abstract Factory Pattern: Implementation



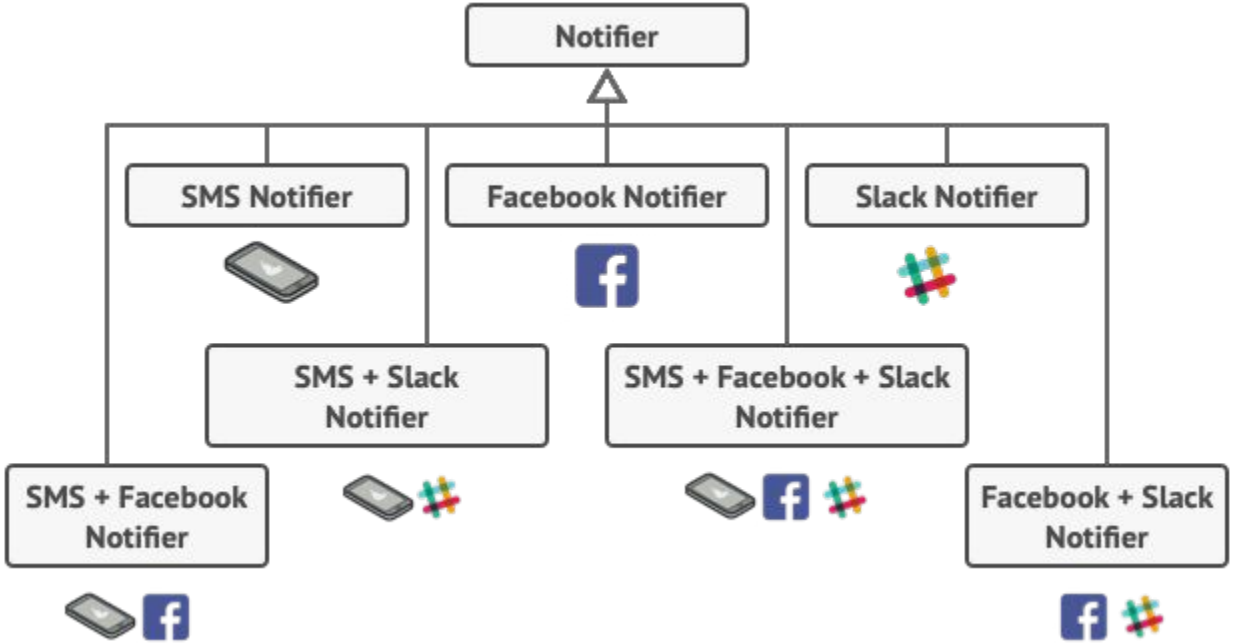
Live Example: Abstract Factory Pattern

- Inside **src/universities**, there is a command-line system for registering students and lecturers from either USYD or UNSW.
  - People involved with UNSW have 7-digit IDs in the form 5XXXXXX.
  - People involved with USYD have 9-digit IDs.
- Apply the Abstract Factory Pattern to improve the quality of the code.

Decorator Pattern: Motivation

- Suppose we have vanilla, chocolate and strawberry ice cream and we were modelling different combinations of scoops as classes. For example,
  - `VanillaIceCream`
  - `VanillaAndStrawberryIceCream`
  - `ChocolateAndStrawberryIceCream`
  - `VanillaChocolateAndStrawberryIceCream`
  - and so on...
- What is the issue with this approach?
  - We would end up with *way* too many classes.
- How can we effectively model classes in these scenarios, where we have many different combinations of functionalities that can be added on top of each other?

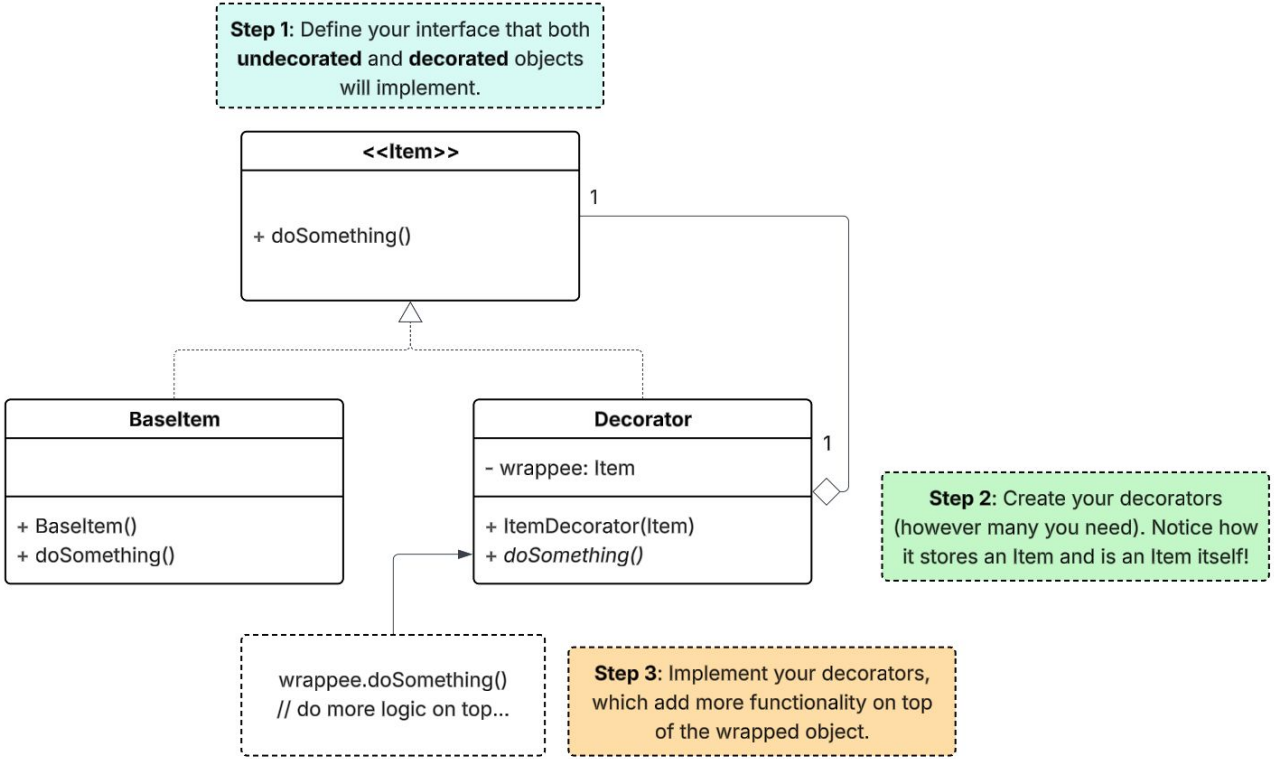
Decorator Pattern: Motivation (more technical example)



Decorator Pattern

- The Decorator Pattern is a (structural) design pattern that allows new behaviours to be attached on top of an existing object.
- The existing object gets stored (**wrapped**) in another object, referred to as a **decorator**. The decorator calls on the original functionality of the wrapped object, then modifies it or adds additional functionality.
  - Importantly, both unwrapped and wrapped objects need to implement a common interface - this allows us to add more decorators onto already decorated objects.
  - In the ice cream example, the unwrapped object could be an ice cream cone, and the decorators would be different scoops of ice cream - we are adding more things on top of the ice cream cone, and they can be added in any order.

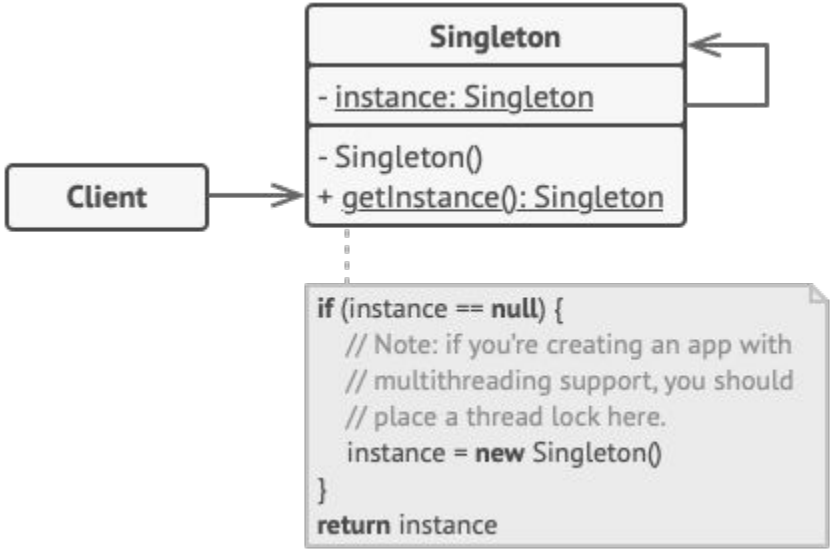
Decorator Pattern Implementation



Live Example: Decorator Pattern

- Develop a system in **src/milktea** to print out a milk tea order, where customers are able to order milk tea and optionally add the following toppings on (for an extra cost):
  - Pearls, costing $0.80.
  - Lychee Jelly, costing $0.90.
  - Milk Foam, costing $1.00.

Singleton Pattern

- The Singleton Pattern is a (creational) design pattern that ensures that only one instance of a class exists at any point in time.
  - This may be desired when a class involves some sort of shared resource, and there are issues with multiple threads concurrently accessing it.
- The implementation is very simple:
  - Make the class constructor **private**.
  - Make the class store a static instance of itself - this is the **singleton**.
  - Only allow clients to 'instantiate' the object through a static method that returns the instance if it exists (if it doesn't exist, instantiate it first).
  - In Java, the `synchronized` keyword can be added to methods to prevent concurrent access from multiple threads.

Singleton Pattern Implementation